

# Explorando o Paralelismo em Random Forests: Desempenho, Fidelidade Preditiva e Eficiência Energética em CPU e GPU

Gabriel L. Bonavina<sup>1</sup>, Estevan T. Santos<sup>1</sup>,  
Marcelo F. Ohno<sup>1</sup>, Vinicius M. Goulart<sup>1</sup>

<sup>1</sup>Universidade Federal de São Paulo (Unifesp)

e-mail: gabriel.bonavina@unifesp.br estevan.santos@unifesp.br

**Abstract.** *Random Forest aggregates decision trees trained on bootstrap samples, evaluating random feature subsets at each split and determining predictions by majority vote. Sorting candidate splits at each internal node dominates training runtime. This paper evaluates sequential C11, multi-core OpenMP, and GPU CUDA implementations across four benchmarks (150 to 581,012 samples). OpenMP accelerates training by  $2.69\times$  to  $7.99\times$ , reducing the energy-delay product (EDP) by up to  $31.2\times$ . In contrast, end-to-end GPU inference reaches  $28.65\times$  speedup, producing zero prediction discrepancies relative to the CPU for the same forest.*

**Resumo.** *O algoritmo Random Forest combina árvores de decisão induzidas sobre amostras bootstrap, avaliando subconjuntos aleatórios de atributos em cada partição e definindo a predição por voto majoritário. A ordenação dos cortes candidatos em cada nó interno constitui o gargalo preponderante do treinamento. Este trabalho compara implementações em C11 sequencial, OpenMP em CPU e CUDA em GPU sobre quatro bases (150 a 581 mil instâncias). O OpenMP acelera o treino de  $2,69\times$  a  $7,99\times$  e o produto energia-atraso (EDP) em até  $31,2\times$ . Na inferência fim a fim, a GPU alcança ganho de  $28,65\times$ , com correspondência preditiva exata à CPU para o mesmo modelo.*

## 1. Introdução

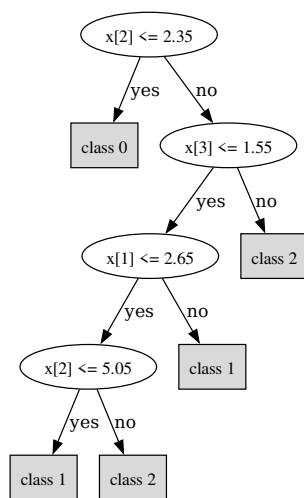
A classificação supervisionada mapeia instâncias com atributos numéricos a classes discretas, com ampla aplicação em diagnóstico médico, visão computacional e sensoria-mento remoto. Entre os métodos baseados em comitês (*ensembles*), o Random Forest [Breiman 2001, Breiman et al. 1984] destaca-se pela alta acurácia preditiva e robustez ao sobreajuste por meio da agregação de múltiplas árvores de decisão.

Contudo, a indução de florestas sobre bases volumosas impõe elevado custo computacional. O algoritmo exibe paralelismo híbrido: as árvores crescem de forma independente, mas a construção de cada nó exige ordenações sucessivas e caminhos de controle irregulares. Este artigo avalia três implementações do algoritmo (C11 sequencial, OpenMP [OpenMP Architecture Review Board 2018] e CUDA [NVIDIA Corporation 2025]) quanto a tempo de execução, acurácia e Produto Energia-Atraso (EDP) [Gonzalez and Horowitz 1996], com medições diretas via Intel RAPL [David et al. 2010] e NVIDIA NVML [NVIDIA Corporation 2024] em bases de 150 a 581.012 instâncias.

## 2. Random Forest

### 2.1. Árvore de decisão

O conjunto de treino compreende amostras  $(x_i, y_i)$ , onde cada vetor  $x_i \in \mathbb{R}^d$  representa uma instância de  $d$  atributos contínuos e  $y_i \in \{1, \dots, K\}$  denota sua classe entre  $K$  categorias. Uma árvore de classificação e regressão (CART) [Breiman et al. 1984] particiona o espaço de atributos recursivamente por meio de testes univariados da forma  $x_f \leq \theta$ , com  $f \in \{1, \dots, d\}$  e  $\theta \in \mathbb{R}$ . Instâncias que satisfazem a desigualdade seguem para o filho esquerdo, e as demais para o direito. A Figura 1 ilustra uma árvore induzida no Iris, em que o nó raiz avalia o atributo 2 com limiar 2,35, e as folhas definem a classe predita.



**Figura 1. Árvore ilustrativa induzida no Iris. Atributos numerados a partir de zero; cada folha devolve a classe majoritária entre os exemplos que a alcançam.**

Em cada nó interno com subconjunto de amostras  $S$ , o corte ideal maximiza a homogeneidade das partições geradas, mensurada pela impureza de Gini. Sendo  $p_k$  a fração empírica de instâncias da classe  $k$  em  $S$ ,  $I_G(S) = 1 - \sum_{k=1}^K p_k^2$ . Um nó puro exibe  $I_G = 0$ , enquanto a divisão equilibrada entre duas classes resulta em  $I_G = 0,5$ . Ao avaliar um candidato que divide  $S$  em  $S_L$  e  $S_R$ , calcula-se a impureza média ponderada  $I_G(S, f, \theta) = \frac{|S_L|}{|S|} I_G(S_L) + \frac{|S_R|}{|S|} I_G(S_R)$ , elegendo-se o corte de menor valor. Na implementação, ordenam-se os valores do atributo  $f$  nas linhas de  $S$  e os limiares posicionam-se no ponto médio entre valores adjacentes distintos. A recursão cessa se o nó for puro, se  $|S| < n_{\min}$ , ou na profundidade máxima  $D$ , atribuindo-se à folha a classe majoritária. Árvores profundas têm alta variância.

### 2.2. Da árvore à floresta

O Random Forest atenua essa variância combinando  $T$  árvores de decisão descorrelacionadas [Breiman 2001] por meio de dois mecanismos estocásticos: (i) *bootstrap aggregating (bagging)* [Breiman 1996], onde cada árvore é induzida sobre  $N$  amostras sorteadas com reposição (cerca de 63,2% das instâncias originais); e (ii) *subespaço aleatório de*

*atributos*, avaliando-se em cada nó apenas  $m \leq d$  atributos sem reposição (tipicamente  $m \approx \lfloor \sqrt{d} \rfloor$ , passível de calibração), o que evita a predominância de atributos fortes.

Na inferência, a amostra percorre cada árvore até uma folha, definindo a predição por voto majoritário simples. Como as árvores não trocam dados durante o crescimento, o laço de indução é embaraçosamente paralelo. O determinismo e a reprodutibilidade estrita decorrem do gerador xorshift32 com semente  $s_t = s + t \cdot 0 \times 9 \text{E} 3779 \text{B} 9 + 1$ , com  $s = 42$  para a partição holdout (80% treino e 20% teste), assegurando que as três versões percorram os nós na mesma pré-ordem e consumam idêntica sequência pseudoaleatória.

### 2.3. Análise do gargalo computacional

O Algoritmo 1 formaliza o fluxo implementado. Três grandezas fundamentam a análise assintótica da complexidade:  $N$  denota o número total de instâncias de treinamento (dimensão de cada réplica bootstrap com reposição);  $S$  representa o subconjunto de amostras que chegam a determinado nó recursivo, sendo  $|S|$  a sua cardinalidade local ( $|S| = N$  no nó raiz e  $|S| < N$  nos nós descendentes); e  $D$  é a profundidade máxima permitida (*max depth*), delimitando o comprimento do caminho da raiz até qualquer folha.

---

**Algoritmo 1:** Fluxo de indução e predição do Random Forest para classificação. O gargalo computacional concentra-se na etapa de ordenação no interior de CRESCER, reiterada para cada nó e atributo avaliado.

---

```

1  procedimento Floresta ( $X, y, T, D, n_{\min}, m$ ) :
2      para  $t \leftarrow 1$  até  $T$  faça
3           $B \leftarrow N$  índices com reposição;           // amostragem bootstrap:  $O(N)$ 
4          raiz de  $t \leftarrow \text{Crescer}(B, 0)$ ;
5      fim
6  procedimento Crescer ( $S, \text{profundidade}$ ) :
7      se  $S$  é puro ou  $|S| < n_{\min}$  ou  $\text{profundidade} = D$  então
8          retorna folha com a classe majoritária;
9      fim
10     sortear  $m$  atributos sem reposição;
11     para cada atributo  $f$  sorteado faça
12         montar os pares (valor de  $f$ , classe) das linhas de  $S$ ;
13         ordenar os pares pelo valor;                     // gargalo:  $O(|S| \log |S|)$ 
14         varrer cortes entre valores vizinhos distintos;
15         guardar o corte de menor Gini ponderado;
16     fim
17     partir  $S$  em  $E$  e  $R$  pelo melhor corte;
18     esquerdo  $\leftarrow \text{Crescer}(E, \text{profundidade} + 1)$ ;    // subdivisão recursiva
19     direito  $\leftarrow \text{Crescer}(R, \text{profundidade} + 1)$ ;
20     retorna o nó interno;
21 procedimento Predizer ( $x$ ) :
22     para cada árvore faça
23         descer até uma folha;                             // travessia  $O(D)$  por árvore
24         somar um voto da classe da folha;
25     fim
26     retorna a classe com mais votos;

```

---

A amostragem bootstrap possui custo  $O(N)$  por árvore, fração desprezível do tempo total. O gargalo computacional reside na rotina CRESCER: para cada um dos  $m$  atributos examinados no nó, as  $|S|$  linhas locais são ordenadas em pares (valor, classe).

Como a varredura subsequente de cortes é linear ( $O(|S|)$ ), o custo dominante por nó é governado pelo termo  $O(m |S| \log |S|)$ .

Os nós próximos à raiz concentram maior custo, pois  $|S| \approx N$ . Embora divisões sucessivas particionem os dados, a soma das instâncias em cada nível de profundidade mantém-se próxima de  $N$ . Limites  $D$  elevados ( $D = 30$ , em Letter e Coverttype) multiplicam as ordenações de grande porte. Em contrapartida, a inferência (PREDIZER) restringe-se a percorrer um caminho de comprimento  $\leq D$  por árvore, com custo leve  $O(D)$  por amostra e acesso somente de leitura em memória.

### 3. Implementação paralela

#### 3.1. OpenMP

No treinamento da CPU multi-core, a diretiva `#pragma omp parallel for` com escalonamento dinâmico (`schedule(dynamic)`) entrega uma árvore por iteração. A arena de nós e o estado do gerador são privativos de cada thread, eliminando contenções de sincronização. O escalonamento dinâmico justifica-se pelo tamanho assimétrico das árvores induzidas. Na predição, `schedule(static)` distribui as linhas igualmente, atuando sobre árvores em modo somente leitura. A versão sequencial em C11 compartilha da mesma lógica em um único fluxo.

#### 3.2. CUDA

Na GPU, cada bloco de 256 threads induz uma árvore. Para bases de menor porte ( $N \leq 1.024$ , Iris e WDBC), os índices e pares (valor, classe) permanecem na memória compartilhada (*shared memory*) do bloco, e a ordenação cooperativa utiliza uma rede bitônica paralela [Batcher 1968] com barreiras `__syncthreads()`. Acima desse limite (Letter e Coverttype), a ordenação bitônica opera na memória global (VRAM) e as árvores são processadas em lotes cujo espaço de trabalho não excede 512 MB, viabilizando a execução em GPUs de 4 GB.

A expansão dos nós emprega uma pilha LIFO: o filho direito é empilhado antes do esquerdo, assegurando que o esquerdo seja processado primeiro, como na recursão em pré-ordem da CPU. A predição lança uma thread por amostra. Nas medições, distingue-se o tempo de kernel do tempo fim a fim (que engloba o tráfego via PCIe).

### 4. Metodologia experimental

Os conjuntos de dados utilizados são Iris [Fisher 1936], Wisconsin Diagnostic Breast Cancer (WDBC) [Street et al. 1993], Letter Recognition [Frey and Slate 1991] e Forest Coverttype [Blackard and Dean 1999]. Os hiperparâmetros de Iris, WDBC e Letter foram calibrados por busca em grade no binário OpenMP, selecionando a configuração mais concisa cuja acurácia média de teste permaneceu a até 0,0025 da melhor observada. O Coverttype foi fixado em 50 árvores, profundidade 30 e  $m = 18$ . As três versões recebem os mesmos parâmetros e sementes. A Tabela 1 resume as dimensões, o holdout (80% treino e 20% teste) e a acurácia.

O hardware compreende uma CPU Intel Core i5-12500H (12 núcleos físicos, sendo 4 de desempenho e 8 de eficiência, 16 threads lógicas) e uma GPU dedicada NVIDIA GeForce RTX 3050 Laptop (4 GB VRAM, arquitetura Ampere, capacidade de

**Tabela 1. Partição holdout de 20% e acurácia de teste. O número mínimo de amostras para partição ( $n_{\min}$ ) é 5 para Iris e WDBC, e 2 para Letter e Covertype. A acurácia na CPU é idêntica nas versões sequencial e OpenMP.**

| Conjunto | Treino  | Teste   | $d$ | $K$ | Árv. | Prof. | $m$ | Acurácia |        |
|----------|---------|---------|-----|-----|------|-------|-----|----------|--------|
|          |         |         |     |     |      |       |     | CPU      | GPU    |
| Iris     | 120     | 30      | 4   | 3   | 300  | 12    | 2   | 0,9333   | 0,9333 |
| WDBC     | 455     | 114     | 30  | 2   | 300  | 6     | 3   | 0,9912   | 0,9912 |
| Letter   | 16.000  | 4.000   | 16  | 26  | 300  | 30    | 4   | 0,9617   | 0,9603 |
| Cover    | 464.809 | 116.203 | 54  | 7   | 50   | 30    | 18  | 0,9599   | 0,9603 |

computação 8.6), sob o Windows 11. O consumo de energia consolidado é dado por  $E = E_{\text{CPU}} + E_{\text{GPU}}$ , monitorando o processador via Intel RAPL em nanojoules e a GPU via NVML em milijoules. A potência média é  $\bar{P} = E/\Delta t$  e o Produto Energia-Atraso é  $\text{EDP} = E \cdot \Delta t$ , onde  $\Delta t$  é o tempo de relógio de parede do processo completo (leitura de dados, treino e inferência). A potência em repouso do sistema foi de 17,08 W ao longo de 3 s prévios. Reporta-se a energia bruta integral, sem descontos.

Para Iris, WDBC e Letter, realizaram-se três execuções após uma rodada preliminar de aquecimento (*warm-up*) descartada, utilizando-se as médias aritméticas. O Covertype teve execução única sem aquecimento. Os tempos de treino e predição vêm dos cronômetros internos do programa. A predição afere treino e teste conjuntamente para cálculo da acurácia global, enquanto a coluna Kernel isola a inferência no teste na GPU. O processo CUDA reavalia a floresta na CPU para checar divergências, computando esse tempo na energia e no EDP, mas excluindo-o do tempo de predição da GPU. Como os sensores atualizam-se na escala de dezenas de milissegundos, execuções inferiores a 0,5 s (Iris) atingem o piso de sensibilidade dos medidores.

## 5. Resultados

Os tempos de execução e respectivos speedups estão consolidados na Tabela 2 (treino) e na Tabela 3 (predição). A Figura 2 sintetiza os speedups de treino, predição e EDP. No treinamento, o OpenMP alcança fatores de aceleração de  $2,69\times$  a  $7,99\times$ . A aceleração da GPU supera o sequencial apenas no WDBC ( $1,63\times$ ). No Covertype, o treinamento demanda 213,81 s no sequencial, 34,49 s no OpenMP e 483,32 s na GPU.

**Tabela 2. Tempo de treinamento e speedup sobre a linha de base sequencial.**

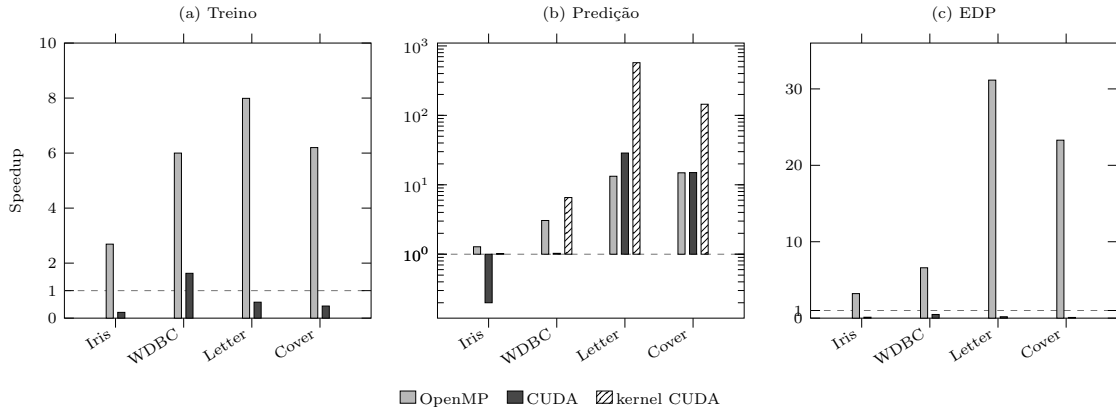
| Conjunto | Seq. (s) | OpenMP (s) | CUDA (s) | $S_{\text{OMP}}$ | $S_{\text{CUDA}}$ |
|----------|----------|------------|----------|------------------|-------------------|
| Iris     | 0,010    | 0,004      | 0,047    | 2,69             | 0,21              |
| WDBC     | 0,130    | 0,022      | 0,079    | 6,00             | 1,63              |
| Letter   | 8,07     | 1,01       | 13,84    | 7,99             | 0,58              |
| Cover    | 213,81   | 34,49      | 483,32   | 6,20             | 0,44              |

Na predição fim a fim, o OpenMP proporciona ganhos graduais de  $1,28\times$  no Iris até  $14,9\times$  no Covertype. A GPU fim a fim destaca-se em volumes elevados, alcançando  $28,7\times$  no Letter (de 1,78 s para 62,2 ms) e  $15,0\times$  no Covertype (897 ms contra 13,45 s). Ao isolar o kernel de teste na GPU, obtêm-se 3,11 ms no Letter e 93,0 ms no Covertype (fatores de 573 e 145 na Tabela 3). Como a contagem sequencial inclui treino e teste (com

**Tabela 3. Tempo de predição sobre treino e teste (em milissegundos) e speedup sobre o sequencial. A coluna Kernel isola a execução da GPU unicamente no teste;  $S_{\text{ker}}$  denota a razão direta Seq./Kernel, equivalente a ganhos amostrais efetivos de cerca de  $115\times$  e  $29\times$  ao se isolar o teste.**

| Conjunto | Seq.   | OMP  | CUDA | Kernel | $S_{\text{OMP}}$ | $S_{\text{CUDA}}$ | $S_{\text{ker}}$ |
|----------|--------|------|------|--------|------------------|-------------------|------------------|
| Iris     | 0,48   | 0,38 | 2,45 | 0,47   | 1,28             | 0,20              | 1,02             |
| WDBC     | 3,76   | 1,23 | 3,65 | 0,57   | 3,06             | 1,03              | 6,55             |
| Letter   | 1.780  | 134  | 62,2 | 3,11   | 13,3             | 28,7              | 573              |
| Cover    | 13.446 | 904  | 897  | 93,0   | 14,9             | 15,0              | 145              |

o treino correspondendo a 80% das amostras), ponderar somente o teste resulta em ganhos efetivos de cerca de 115 e 29 vezes, sem transferências. O tempo fim a fim constitui o indicador realista para sistemas em produção.



**Figura 2. Speedup obtido em relação à implementação sequencial de referência. A linha tracejada horizontal demarca a paridade unitária ( $1\times$ ). Em (b), a escala é logarítmica (base 10) com barras ancoradas na paridade unitária ( $1\times$ ), e a série hachurada retrata o tempo de kernel da GPU no teste comparado ao tempo total sequencial. Em (a) e (c), essa subdivisão não se aplica. O registro de energia do conjunto Iris situa-se no piso de resolução dos sensores.**

A Tabela 4 reúne o consumo de energia e o speedup do EDP ( $S_{\text{EDP}} = E_{\text{seq}} \Delta t_{\text{seq}} / E_{\text{par}} \Delta t_{\text{par}}$ ). No Letter, o OpenMP dissipa 41,9 J contra 162 J do sequencial e 560 J da CUDA, produzindo um speedup de EDP de  $31,2\times$ . No Covertype, os consumos são de 903 J (OpenMP), 3.811 J (sequencial) e 19.297 J (CUDA), gerando speedup de EDP de  $23,3\times$  para o OpenMP e  $0,09\times$  para a CUDA. A potência média no Letter e no Covertype situou-se em 16,3 W no sequencial, 34,0 W e 21,3 W no OpenMP, e 35,8 W e 38,2 W na CUDA. Assim, o speedup de EDP da GPU permaneceu abaixo de 1 em todos os conjuntos ( $0,09\times$  a  $0,48\times$ ).

Em acurácia, sequencial e OpenMP coincidem perfeitamente. A GPU reproduz a CPU em Iris e WDBC, exibindo divergências residuais na terceira casa decimal em Letter e Covertype (0,9603 contra 0,9617 e 0,9603 contra 0,9599). O binário CUDA registrou zero divergências de predição entre GPU e CPU para a mesma floresta treinada em todas as 30, 114, 4.000 e 116.203 amostras de teste. Portanto, o treinamento na GPU gera árvores sutilmente distintas da CPU devido à ordem em que empates na impureza de Gini

**Tabela 4. Consumo de energia integral do processo e speedup de EDP sobre o sequencial. Os registros do conjunto Iris situam-se na resolução mínima dos sensores RAPL e NVML.**

| Conjunto | Seq. (J) | OpenMP (J) | CUDA (J) | $S_{EDP}^{OMP}$ | $S_{EDP}^{CUDA}$ |
|----------|----------|------------|----------|-----------------|------------------|
| Iris     | 8,16     | 3,45       | 10,6     | 3,21            | 0,12             |
| WDBC     | 6,39     | 3,62       | 10,4     | 6,59            | 0,48             |
| Letter   | 162      | 41,9       | 560      | 31,2            | 0,18             |
| Cover    | 3.811    | 903        | 19.297   | 23,3            | 0,09             |

são desfeitos (varredura linear na CPU versus redução paralela bitônica no bloco CUDA). Em contrapartida, quando uma mesma floresta é avaliada em ambas as arquiteturas, a inferência exhibe determinismo rigoroso e correspondência perfeita.

## 6. Discussão

O treinamento favorece a arquitetura de CPU com OpenMP. A combinação de recursão eficiente, caches privativas L1/L2 e previsão de saltos por núcleo minimiza as latências de ordenação e corte. A independência entre árvores viabiliza um speedup de  $7,99\times$  no Letter sobre 16 threads lógicas. O teto decorre da variação no tamanho das árvores e da disputa no barramento de memória durante as ordenações paralelas. Na GPU, o treino supera o sequencial apenas no WDBC ( $1,63\times$ ), cuja base de 455 instâncias cabe na memória compartilhada rápida. Em Letter e Coverttype, a necessidade de sincronizar threads a cada estágio da rede bitônica em memória global, somada às árvores profundas ( $D = 30$ ) com nós assimétricos, introduz severa divergência de warp e custo de sincronização, tornando a execução até 2,3 vezes mais lenta que um núcleo sequencial ( $0,44\times$  no Coverttype).

Na inferência, a GPU manifesta sua superioridade arquitetural. Uma vez construída a floresta, cada amostra de teste é avaliada de forma independente sobre estruturas estáticas de leitura. A ausência de contenções de escrita maximiza a ocupação dos multiprocessadores. Em conjuntos volumosos (Letter e Coverttype), o rendimento massivo da GPU amortiza com folga o tráfego via PCIe, atingindo speedups fim a fim de  $28,7\times$  e  $15,0\times$ . No Iris (30 instâncias de teste), a sobrecarga do protocolo prepondera, resultando em speedup de  $0,20\times$ . O OpenMP surge como alternativa intermediária equilibrada ( $1,28\times$  a  $14,9\times$ ), com latência nula de transferência de dados.

A métrica de EDP acompanha o tempo de treino, que domina o processo nos conjuntos maiores. A RTX 3050 Laptop dissipa entre 35 W e 38 W continuamente durante a execução CUDA. Sem compressão temporal, essa potência mantida por centenas de segundos degrada o produto energia-atraso. O OpenMP, embora aumente a potência instantânea (34 W no Letter), encurta o tempo em proporção bem superior, gerando economia líquida de energia e ganhos de EDP de até  $31,2\times$ . A checagem auxiliar na CPU adiciona carga secundária na energia CUDA (cerca de 13 s diante de 483 s de treino no Coverttype), sem alterar as conclusões centrais.

## 7. Conclusão

Neste trabalho, avaliou-se o compromisso entre tempo, acurácia preditiva e eficiência energética na paralelização do Random Forest em CPU e GPU sobre quatro conjuntos de dados heterogêneos.

Os resultados demonstram que o treinamento do algoritmo é significativamente mais eficiente em CPU multi-core com OpenMP, alcançando acelerações de até  $7,99\times$  e ganhos de EDP de até  $31,2\times$ . A indução em GPU é penalizada por divergência de fluxo e sincronizações em memória global, sendo vantajosa apenas quando as amostras cabem na memória compartilhada rápida do bloco. Por outro lado, a inferência consolida-se como o domínio ótimo da GPU, onde o paralelismo massivo por amostra atinge ganhos fim a fim de até  $28,65\times$ , com correspondência exata de predições com a CPU para um mesmo modelo induzido.

Conclui-se que o arranjo ótimo para Random Forests consiste em um modelo heterogêneo cooperativo: induzir o comitê de árvores em CPU multi-core via OpenMP e descarregar a inferência em lote para a GPU via CUDA. Trabalhos futuros podem avaliar representações aproximadas por histogramas na GPU e sobreposição assíncrona de transferências na predição.

## Referências

- Batcher, K. E. (1968). Sorting networks and their applications. In *Proceedings of the AFIPS Spring Joint Computer Conference*, pages 307–314.
- Blackard, J. A. and Dean, D. J. (1999). Comparative accuracies of artificial neural networks and discriminant analysis in predicting forest cover types from cartographic variables. *Computers and Electronics in Agriculture*, 24(3):131–151.
- Breiman, L. (1996). Bagging predictors. *Machine Learning*, 24(2):123–140.
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1):5–32.
- Breiman, L., Friedman, J. H., Olshen, R. A., and Stone, C. J. (1984). *Classification and Regression Trees*. Wadsworth.
- David, H., Gorbato, E., Hanebutte, U. R., Khanna, R., and Le, C. (2010). RAPL: Memory power estimation and capping. In *Proceedings of the ACM/IEEE International Symposium on Low Power Electronics and Design*, pages 189–194.
- Fisher, R. A. (1936). The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2):179–188.
- Frey, P. W. and Slate, D. J. (1991). Letter recognition using Holland-style adaptive classifiers. *Machine Learning*, 6(2):161–182.
- Gonzalez, R. and Horowitz, M. (1996). Energy dissipation in general purpose microprocessors. *IEEE Journal of Solid-State Circuits*, 31(9):1277–1284.
- NVIDIA Corporation (2024). *NVIDIA Management Library (NVML)*. API de telemetria do driver gráfico.
- NVIDIA Corporation (2025). *CUDA C++ Programming Guide*.
- OpenMP Architecture Review Board (2018). *OpenMP Application Programming Interface*. Version 5.0.
- Street, W. N., Wolberg, W. H., and Mangasarian, O. L. (1993). Nuclear feature extraction for breast tumor diagnosis. In *Biomedical Image Processing and Biomedical Visualization*, volume 1905, pages 861–870. SPIE.